BattingEdge: Frontend Development & Integration Report

Project: AI-Powered Cricket Shot Analysis & Biomechanics Coaching System

Phase: UI/UX Implementation & System Integration

Date: December 8, 2025

---

1. Executive Summary

Following the successful validation of the Backend API (V8p Model), the project focus shifted to developing a user-friendly web interface. We implemented a modern, responsive **React Application** that serves as the visual dashboard for the system.

Key achievements in this phase include:

1. **Dashboard UI:** A dark-themed, "expensive-looking" interface inspired by modern SaaS platforms (Vercel/Stripe).
2. **Full Integration:** Seamless connection between the React Frontend and Python FastAPI Backend.
3. **Visual Polish:** Implementation of H.264 video streaming, animated score gauges, and interactive checklists.
4. **Reporting:** Automated generation of professional PDF coaching reports with grading (A/B/C) and specific corrective advice.

---

2. Frontend Technology Stack

- **Core Framework: React 18** (via **Vite** for fast build times).
- **Styling: Tailwind CSS** (Utility-first CSS for rapid, responsive design).
- **Animations: Framer Motion** (Smooth transitions, progress bars, and entry animations).
- **HTTP Client: Axios** (For communication with the Python Backend).
- **Notifications: React Hot Toast** (Real-time alerts for upload status).
- **Icons: Lucide React** (Clean, modern SVG icons).

---

3. Application Architecture & Components

A. Landing & Upload Interface (UploadPage.jsx)

- **Hero Section:** Features a dynamic text introduction with gradient styling tailored for both Light and Dark modes.
- **Upload Zone:** A drag-and-drop area that validates file types (.mp4, .avi) and sizes (<100MB). It handles the **API Chaining**:
    1. POST /upload (Sends file -> Gets ID).
    2. POST /analyze (Triggers AI).
    3. Redirects to /result/{id}.

B. Analysis Dashboard (ResultPage.jsx)

This is the core user interface, split into a **Grid Layout**:

1. **Video Player (Left Panel):**
    - Streams the processed video with the **Skeleton Overlay** and **HUD**.
    - Includes a "Download" overlay button.
2. **AI Confidence (Left Panel):**
    - Animated horizontal bars showing the probability of every shot type (Drive, Pull, Cut, Sweep).
3. **Score Card (Right Panel):**
    - Displays the **Overall Technique Score** (0-100).
    - Assigns a letter grade (**A/B/C**) based on the score.
    - Dynamic color coding (Green/Yellow/Red).
4. **Key Improvements (Right Panel):**
    - A dedicated "Alert Box" that only appears if specific errors are detected (e.g., "Head falling over").
5. **Detailed Checklist (Right Panel):**
    - An expandable list of biomechanical metrics (Elbow Angle, Feet, Hips).
    - Shows **Measured Value** vs **Target Benchmark** (e.g., "110° / Target: 120°-140°").
    - Expands on click to show specific **Coaching Advice**.

---

4. Backend Integration Updates

To support the rich UI, specific upgrades were made to the backend logic.

A. Video Codec Fix (inference.py)

- **Problem:** The original OpenCV video writer used mp4v, which modern browsers (Chrome/Edge) cannot play natively.
- **Solution:** Switched the video writer to use the **avc1 (H.264)** codec.
- **Result:** Processed videos now play instantly in the React video player without needing download.

B. Data Persistence Fix (database.py)

- **Problem:** The UI was showing empty analysis sections because the database was discarding the text summaries.
- **Solution:** Updated update_analysis_result() to serialize and save the **entire** form_analysis dictionary (including summary and key_improvements) into the database, not just the raw checks.

C. PDF Report Engine (report.py)

- **Upgrade:** Redesigned the PDF generation to match the UI's quality.
    - Added a graphical header.
    - Added a "Grade Card" visual.
    - Added a specific "Areas for Improvement" section highlighted in red.
    - Added a "Target vs Actual" column in the metrics table.

---

5. Operational Guide

To run the full system for your defense presentation, follow these steps exactly.

Prerequisites

Ensure you have two terminal windows open.

Step 1: Start the Backend (Brain)

1. Navigate to the project root:

PowerShell

```
cd D:\Users\Anoshia\BattingEdge_FYP
```

2. Activate the environment:

PowerShell

```
.\venv\Scripts\Activate.ps1
```

3. Launch the API server:

PowerShell

```
uvicorn backend.main:app --reload
```

Wait until you see: INFO: Application startup complete.

Step 2: Start the Frontend (Face)

1. Open a new terminal and navigate to the frontend:

PowerShell

```
cd D:\Users\Anoshia\BattingEdge_FYP\frontend
```

2. Launch the React development server:

PowerShell

```
npm run dev
```

3. **Ctrl + Click** the link shown (usually http://localhost:5173) to open the app in your browser.

---

6. Conclusion

The BattingEdge system is now a complete **End-to-End Solution**.

- **Input:** Users upload raw cricket videos via a modern web interface.
- **Processing:** The backend autonomously detects the batsman, classifies the shot (78% accuracy), and analyzes biomechanics using the V8p logic.
- **Output:** The user receives:
    1. A visual overlay video (Skeleton + HUD).

2. A detailed interactive dashboard with grades and targets.
3. A downloadable, professional PDF coaching report.

The system handles errors gracefully, supports Light/Dark modes, and provides actionable feedback, fulfilling all requirements for the Mid-Year Defense.